

HR Hiring Analytics — Full Project Documentation

1. Data Cleaning & Data Pre-processing

1.1 Importing Modules

All essential Python libraries were imported at the beginning of the project to support data manipulation, visualization, and ML modeling:

- **pandas** — for DataFrame operations and tabular data handling
- **numpy** — for numerical computations
- **matplotlib & seaborn** — for data visualization and plotting
- **matplotlib.ticker** — for formatting axis labels (e.g., percentage format)
- **warnings** — to suppress unnecessary warnings during execution

1.2 Data Ingestion

The raw dataset HR_HIRING_ANALYTICS.csv was loaded using `pd.read_csv()`. Since the original column names were in **ALL CAPS with underscores**, a **column mapping dictionary** was created to rename them into a more readable, consistent format (e.g., JOINING_STATUS → Status, REX_IN_YRS → Experience_years). Additionally:

- **Accidental spaces** in column names were stripped using `.str.strip()`
- A **Sno (Serial Number)** column was inserted at position 0 if not already present, to serve as a row index reference
- Final column names were verified and printed

1.3 Column Name Meaning

Each column was documented with its real-world business meaning to understand what it represents:

<u>Column</u>	<u>Meaning</u>
Candidate_ID	Unique ID for each candidate
DOJ_Extended	Whether Date of Joining was extended
Acceptance_days	Days taken to accept the offer
Notice_Period	Notice period in months
Offered_Band	Salary band offered (D1, D2, E1, etc.)
Expected_CTC_hike_pct	Hike % expected by candidate
Offered_CTC_hike_pct	Hike % offered by the company
CTC_diff_pct	Difference between expected and offered hike
Joining_Bonus	Whether a joining bonus was given
Candidate_relocate_actual	Whether candidate actually relocated
Gender	Gender of the candidate
Candidate_Source	How candidate was sourced (referral, portal, etc.)
Experience_years	Years of experience
Age	Age of candidate
LOB	Line of Business (department)
Domicile_Name	Candidate's home state
Location_Name	Job location city
Region_Name	Geographic region (North, South, etc.)
Status	Target Variable — Joined or Not Joined

1.4 Column Name Changing

All raw column names (CAPS format from CSV) were renamed to a clean, readable format using a `column_mapping` dictionary with `df.rename()`. This made the code more readable throughout the project.

1.5 Category Columns

➔ Number of Categorical Columns = 15

Categorical columns were identified and examined for:

- **Unique values** — to understand cardinality
- **Value counts** — to spot dominant categories and rare values
 - **Inconsistencies** — typos, mixed cases, extra spaces

Columns identified as categorical:

DOJ_Extended, Offered_Band, Joining_Bonus, Candidate_relocate_actual, Gender, Candidate_Source, LOB, Domicile_Name, Location_Name, Region_Name, Status

1.6 Binary Values Columns

Binary columns (Yes/No type) were separately examined:

- DOJ_Extended → Yes / No
- Joining_Bonus → Yes / No
- Candidate_relocate_actual → Yes / No
 - Gender → Male / Female

These were later encoded as **0 and 1** during Feature Engineering.

1.7 Numerical Columns

➔ Number of Numerical Columns = 11

Numerical columns were identified and their statistical summaries (min, max, mean, std) were reviewed:

- Acceptance_days, Notice_Period, Expected_CTC_hike_pct, Offered_CTC_hike_pct, CTC_diff_pct, Experience_years, Age

These were checked for **realistic ranges**, **outliers**, and **skewness**.

1.8 Treatment of Duplicate Values

The dataset was checked for **fully duplicate rows** using `df.duplicated().sum()`. Any duplicate records found were dropped to avoid bias during model training.

1.9 NULL Values Treatment

All columns were checked for missing values using `df.isnull().sum()`. Appropriate treatments were applied:

- **Numerical columns** → filled with **median** (robust to outliers)
- **Categorical columns** → filled with **mode** (most frequent value)
- Columns with very high missing % were evaluated for dropping

2. EDA (Exploratory Data Analysis)

2.1 Moving Status Column to End

The target column Status was moved to the **last position** of the DataFrame using `df.pop()` + `df.insert()` for better readability and convention — keeping features on the left and target on the right.

2.2 Outlier Treatment

Outliers were detected using the **IQR (Interquartile Range)** method:

- **IQR = Q3 – Q1**
- **General Rule :**
 - Lower Bound = $Q1 - 1.5 \times IQR$
 - Upper Bound = $Q3 + 1.5 \times IQR$
- But in our project if Tukey Fence Coefficient = 1.5 then the difference between the upper and lower bound and Maximum and Minimum value is very big.
- So we have used Tukey Fence Coefficient Value 3 inspite of 1.5.
- **Our Rule :**
 - Lower Bound = $Q1 - 3 \times IQR$
 - Upper Bound = $Q3 + 3 \times IQR$

Outliers were **capped (Winsorization)** — extreme values were clipped to the lower/upper bound rather than dropped, preserving data volume. Columns treated: Acceptance_days, Expected_CTC_hike_pct, Offered_CTC_hike_pct, CTC_diff_pct, Experience_years, Age

2.3 Treatment of Date_of_Interview Column

The Date_of_Interview column was in object/string format and was converted to **datetime64** using `pd.to_datetime()`. From it, new time-based features were extracted:

- Interview_Year
- Interview_Month
- Interview_Day
- Interview_DayOfWeek

These derived features helped capture **seasonal hiring patterns**. The original Date_of_Interview column was later dropped in Feature Engineering as it was not directly usable by ML models.

2.4 Treatment of Postal_Code Column

Postal_Code was in **object (string)** format. It was examined for:

- Missing values
- Invalid/inconsistent codes
- Since it had very low ML relevance and high cardinality, it was treated/dropped during Feature Engineering

2.5 Distribution Plots of Numerical Columns

Histograms and KDE (Kernel Density Estimate) plots were plotted for all numerical columns to understand their distribution:

- Checked for **skewness** (left/right skewed vs normal)
- Identified **bimodal or multimodal** distributions
- Compared distributions for **Joined vs Not Joined** groups to spot separability

2.6 Plots of Categorical Columns

Count plots and bar charts were created for all categorical columns:

- Each category was plotted with **hue = Status** to see joining rate per category
- Helped identify which categories had high/low joining ratios

- Example insight: Certain LOB types or Candidate_Source channels had significantly higher Not Joined rates

2.7 All Columns vs Status — Statistical Analysis

A **statistical comparison** was done between Joined (0) and Not Joined (1) groups for every column:

- **Numerical columns** → `groupby('Status').mean()` to compare averages
- **Categorical columns** → cross-tabulation and proportion analysis
- This helped identify the **most influential features** driving the target variable

2.8 Correlation Heatmap

A **Pearson Correlation Heatmap** was plotted for all numerical columns to:

- Identify **highly correlated features** (multicollinearity)
- Understand relationship between features and Status
- Features with very high inter-correlation (e.g., > 0.90) were flagged for potential removal to avoid redundancy in ML models

3. Feature Engineering

3.1 Drop Unnecessary Columns

The following columns were dropped before ML as they provide **no predictive value**:

<u>Column</u>	<u>Reason</u>
Sno	Just a serial number / row index
Candidate_ID	Unique primary key — no ML pattern
Location_ID	Location_Name already present
Region_Id	Region_Name already present
Domicile_Id	Domicile_Name already present
LOB_Id	LOB already present
Date_of_Interview	Low contribution; derived features already extracted
Postal_Code	High cardinality, low relevance

3.2 Encoding of Categorical Columns

Since ML models only understand numbers, all categorical columns were encoded:

Binary Encoding

Columns with only 2 values (Yes/No) were encoded as **0 and 1**:

- DOJ_Extended → No=0, Yes=1
- Joining_Bonus → No=0, Yes=1
- Candidate_relocate_actual → No=0, Yes=1
- Gender → Male=0, Female=1

Target Encoding

Status (target column) was encoded:

- Joined → 0
- Not Joined → 1

Ordinal Encoding

Offered_Band has a **natural order** (D1 < D2 < D3 < E1 < E2), so it was encoded with ordered numeric values maintaining the hierarchy.

One-Hot Encoding (OHE)

High-cardinality nominal columns with no inherent order were One-Hot Encoded (creating dummy binary columns per category):

- LOB, Domicile_Name, Location_Name, Region_Name, Candidate_Source

This resulted in a final feature set of **43 columns**.

3.3 3.3 Scaling / Normalization

- Numerical features were scaled based on their skewness values.
- Columns with skewness > |0.5| were scaled using MinMaxScaler, as they exhibited significant skew and were not normally distributed.
- Columns with skewness ≤ |0.5| were scaled using StandardScaler, as they were relatively symmetrical and closer to a normal distribution.
- **MinMaxScaler** — Transforms features to a range of [0, 1]. Applied to: Acceptance_days, Notice_Period, Expected_CTC_hike_pct, Offered_CTC_hike_pct, Experience_years
- **StandardScaler** — Transforms features to have mean = 0 and std = 1. Critical for distance-based models like KNN and Logistic Regression. Applied to: CTC_diff_pct, Age
- Note: Scaling was applied only to continuous numerical columns — encoded binary and categorical columns were excluded.

3.4 Final Dataset Prep for ML

- **X** (features) = all columns except Status → shape: **(8995, 43)**
- **y** (target) = Status column → value counts: 0 (Joined): 7313, 1 (Not Joined): 1682
- Class imbalance ratio ~**4.3 : 1** confirmed — need SMOTE

4. Machine Learning

4.1 Train-Test Split

The dataset was split into **80% train / 20% test** using `train_test_split`:

- `stratify=y` was used to **maintain class proportion** in both splits
- `random_state=42` for reproducibility

X_train : (7196, 43)	X_test : (1799, 43)	y_train : {0: 5850, 1: 1346}	y_test : {0: 1463, 1: 336}
----------------------	---------------------	------------------------------	----------------------------

4.2 SMOTE (Synthetic Minority Oversampling Technique)

Applied **only on training data** (never on test) to fix class imbalance:

- Generates **synthetic samples** for class 1 (Not Joined) by interpolating between existing minority samples
- After SMOTE: {0: 5850, 1: 5850} — perfectly balanced
- X_train shape after SMOTE: **(11700, 43)**
Test set was kept untouched to simulate real-world distribution

4.3 Model Training — 6 Models via For Loop

Six ML models were trained using a **for loop** on `X_train_sm`, `y_train_sm` (SMOTE-balanced), and evaluated on `X_test`, `y_test` (original distribution):

#	Model	Type
1	Logistic Regression	Linear
2	Decision Tree	Tree-based
3	Random Forest	Ensemble (Bagging)
4	Naive Bayes	Probabilistic
5	KNN	Distance-based
6	XGBoost	Ensemble (Boosting)

4.4 Evaluation Metrics (Per Model)

For each model, the following metrics were computed and printed:

Metric	What It Measures
Accuracy	% of total correct predictions
Precision	Of all predicted Not Joined, how many actually didn't join
Recall	Of all actual Not Joined, how many were correctly caught
F1 Score	Harmonic mean of Precision and Recall
ROC-AUC	Model's ability to separate classes across all thresholds
Confusion Matrix	TP, TN, FP, FN breakdown
Classification Report	Full per-class metrics

4.5 Summary Table

All model results were stored in a list of dictionaries, converted to a DataFrame, and sorted by **F1 Score (descending)** for a clean final comparison:

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.6315	0.2829	0.6339	0.3912	0.7000
XGBoost	0.8205	0.5348	0.2976	0.3824	0.7593
Random Forest	0.8010	0.4542	0.3244	0.3785	0.7460
Naive Bayes	0.3552	0.2220	0.9792	0.3619	0.6031
KNN	0.6237	0.2575	0.5387	0.3484	0.6328
Decision Tree	0.7293	0.3127	0.3750	0.3410	0.5914

4.6 Best Model — XGBoost

Based on **Accuracy (82%)**, **ROC-AUC (0.76)**, and **Precision (53%)**, XGBoost is the **best performing model**.